

PCOLAB06 - rapport

Auteur : Mauro Santos, Gabriel Bader

Overview

Ce projet consiste à optimiser la multiplication de matrices carrées ($C = A \cdot B$) en utilisant **plusieurs threads** et un **moniteur de Hoare**.

On découpe la charge de travail de la multiplication en **blocs** et on les stock dans le buffer, puis des workers viennent prendre le job.

Le but pédagogique de ce labo est donc de pratiquer les notions de multi-threading et les moniteurs de Hoare.

Choix de conception

Découpage du projet

La modélisation/implémentation de ce projet se fait comme ceci :

- Le multiplicateur multi-thread (`threadedmatrixmultiplier.h`)

C'est le coeur du labo :

- `ThreadedMatrixMultiplier<T>` : crée les threads au constructeur.
- `Buffer<T>` : file de jobs + synchronisation via moniteur de Hoare.
- `ComputeParameters<T>` : paquet de paramètres ($A, B, C + x, y + blockSize$).
- Les tests (`main.cpp + multipliertester.h + multipliertestedtester.h`)

Ce sont les test qui valident notre implémentation.

Moniteur de Hoare (le buffer)

On a plusieurs threads workers qui tournent en boucle et demandent du travail.

On centralise l'état partagé dans un seul endroit : **la file de jobs**.

On a une classe `Buffer` .

L'idée :

- Le thread qui appelle `multiply()` est le **producteur** (il push les jobs).
- Les workers sont des **consommateurs** (ils pop un job et le calcule).

Avec les méthodes clé :

- `sendJob(params)` : push dans la queue + `signal(notEmpty)` .

- `getJob(parameters)` : fonction bloquante `wait(notEmpty)` qui attend jusqu'à avoir un job
- `jobFinished()` : décrémente les jobs "en cours" et si plus rien → `signal(jobsComplete)` .
- `waitJobs()` : fonction bloquante qui attend que tous les jobs soient finis `wait(notBusy)` .
- `waitFree()` : fonction bloquante qui attend que le buffer soit totalement idle `wait(jobsComplete)` .
- `stop()` : active le flag `isStopped` et réveille tout le monde.

Pourquoi Hoare et pas juste un mutex ?

Parce qu'on a besoin de conditions (par exemple pour savoir si les jobs sont finis) -> Moniteur de Hoare bien.

Implementation de la concurrence

lecture

Dans `doJob` , on lit en boucle :

```
value += params.A->element(i, y) * params.B->element(x, i);
```

Donc A et B sont des ressources **lecture seule** pendant la multiplication.
Plusieurs threads peuvent donc les lire simultanément sans problème.

écriture

Chaque worker écrit dans C :

```
params.C->setElement(x,y,value);
```

Normalement si deux threads écrivent la dans même case, on aurait un problème de concurrence.

Notre choix de conception pour éviter ça :

un job = un bloc de C, donc deux threads ne écrivent jamais sur la même zone.

Concrètement : `multiply()` crée des jobs avec un (x,y) de départ et un `blockSize` , et `doJob` calcule toutes les cases du bloc.

Résultat : pas besoin d'un mutex global sur `C` (on garde plus de parallélisme).

Le buffer

Le buffer est une ressource partagée modifiée par :

- le producteur (`push_back`)
- les workers (`pop_front`)

- les compteurs (`nbJobsDispatched` , etc.)

Ici producers ET consumers sont des “rédacteurs” de la ressource → d'où le moniteur pour protéger l'accès.

Réentrance de `multiply()`

Le labo demande que `multiply()` soit réentrante (test `MultiplierThreadedTester`).

Notre choix pour éviter que les jobs de 2 multiplications se mélangent :

- au début de `multiply(...)` on appelle `buffer.waitForFree()`

Donc si un autre thread appelle `multiply()` pendant qu'un calcul est en cours, il attend que le buffer soit totalement vide.

Tests

Pour l'implémentation des tests nous nous sommes basés sur ce qui existait déjà, nous nous sommes dit qu'il faudrait ajouter quelques tests pour être plus sûr de notre implémentation.

Réentrance

- `ReenteringWith3` : 3 threads appellent `multiply()` l'un après l'autre sur la même instance.

on test que l'état de notre queue reste cohérent

découpage impair

- `OddNumber`

Test avec `matrixSize/nbBlocksPerRow` pas rond (501 / 7). On gère ça en adaptant `blockSize` sur la dernière ligne/colonne + un check de limite dans `doJob` .

Arrêt en plein calcul

- `StopMidMultiplication`

On démarre une multiplication dans un thread, on attend un moment, puis on détruit le multiplicateur.

Tests de `multiply` en parallèle

- `ThreeMultiplySameTime` : 3 threads appellent `multiply()` au même temps sur la même instance

Si on avait un état partagé mal protégé, on verrait des erreurs de calcul ou un interblocage.

Threads en trop

- `UnusedThreads`

On met plus de threads que de jobs utiles.

Les threads en trop doivent juste dormir sur `wait(notEmpty)` .

Conclusion

Ce labo nous a permis de bien pratiquer les notions théoriques comme vu à l'introduction.

Nous sommes satisfait du résultat et nos test montrent que selon nous notre implémentation est fonctionnel.